Chapter 13: Quadratic Programming

Algorithms for Optimization, 2nd Edition

Kochenderfer & Wheeler (2026)

MIT Press

Lecture Slides

Chapter Overview

Outline

What Is a Quadratic Program?

Quadratic Program (General Form) — Eq. (13.1)

$$\underset{\mathbf{x}}{\text{minimize}} \quad \frac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{q}^\top \mathbf{x} \quad \text{subject to} \quad \mathbf{C}_{\text{LE}}\mathbf{x} \leq \mathbf{d}_{\text{LE}}, \quad \mathbf{C}_{\text{EQ}}\mathbf{x} = \mathbf{d}_{\text{EQ}}$$

$\mathbf{Q} \in \mathbb{R}^{n \times n}$	symmetric cost matrix (Hessian of objective)
$\mathbf{q} \in \mathbb{R}^n$	linear cost vector (gradient term)
$\mathbf{C}_{\text{LE}}, \mathbf{d}_{\text{LE}}$	linear inequality constraints
$\mathbf{C}_{\text{EQ}}, \mathbf{d}_{\text{EQ}}$	linear equality constraints

- $\frac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x}$ contributes quadratic components $\frac{1}{2}Q_{ii}x_i^2$ and product components $(\frac{1}{2}Q_{ij} + \frac{1}{2}Q_{ji})x_i x_j$ for $i < j$
- **Connection to Newton's method:** second-order Taylor approximation about $\mathbf{x}_{\text{ref}}$ gives $\mathbf{Q} = \nabla^2 f(\mathbf{x}_{\text{ref}})$, $\mathbf{q} = \nabla f(\mathbf{x}_{\text{ref}})$
- QP = Newton's method subject to linear constraints

Convex QP $\Leftrightarrow$ Least-Squares Problem

Conversion via Cholesky — Eq. (13.2)

When $\mathbf{Q}$ is **positive definite**, factor $\mathbf{Q} = \mathbf{U}^\top \mathbf{U}$ (Cholesky). Then the QP becomes the *least-squares problem*:

$$\underset{\mathbf{x}}{\text{minimize}} \ \|\mathbf{A}\mathbf{x} - \mathbf{b}\| \quad \text{s.t.} \quad \mathbf{C}_{\text{LE}}\mathbf{x} \leq \mathbf{d}_{\text{LE}},\ \mathbf{C}_{\text{EQ}}\mathbf{x} = \mathbf{d}_{\text{EQ}}$$

with $\mathbf{A} = \mathbf{U}$ and $\mathbf{b} = -\mathbf{U}^{-\top}\mathbf{q}$.

Transformation chain (Fig. 13.1):

Figure 13.1 — Transformation chain for QP with positive-definite $\mathbf{Q}$

Key observations:

- Feasible set = convex polytope (same as LP)
- Solution need *not* lie at a vertex
- May be interior, boundary, infeasible, or unbounded

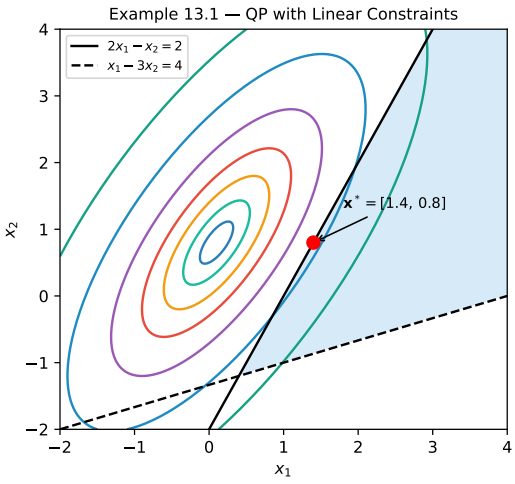
Example 13.1 — QP with Elliptical Contours

Problem

$$\underset{x_1,x_2}{\text{minimize}} \left\| \begin{bmatrix} 2 & 1 \\ -4 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right\|$$

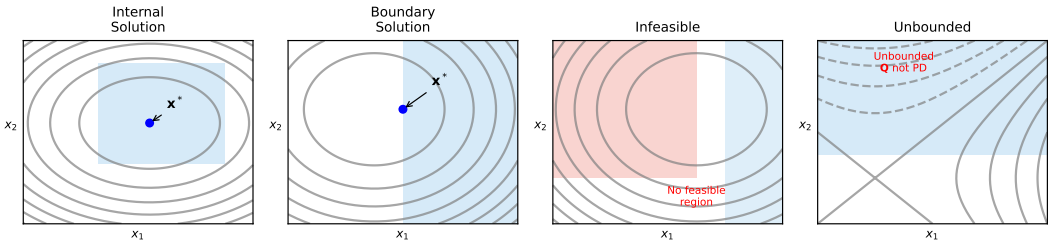
$$\text{s.t.} \quad 2x_1 - x_2 \geq 2, \quad x_1 - 3x_2 \leq 4$$

- Objective forms elliptical contours
- Center of ellipses may lie *outside* feasible set
- **Solution:** $\mathbf{x}^* = [1.4,\ 0.8]^\top$
- Solution on boundary, not at vertex



Four Solution Cases for a QP

Figure 13.2 — QP Solution Cases



- **Internal:** unconstrained minimizer is inside feasible set
- **Boundary:** minimizer is on the boundary of the feasible set
- **Infeasible:** feasible set is empty
- **Unbounded:** **Q** is not positive definite $\Rightarrow$ may be unbounded

Outline

Problem Statement — Eq. (13.3)

$$\underset{\mathbf{x}}{\text{minimize}} \ \|\mathbf{A}\mathbf{x} - \mathbf{b}\|, \quad \mathbf{A} \in \mathbb{R}^{m \times n},\ \mathbf{x} \in \mathbb{R}^n,\ \mathbf{b} \in \mathbb{R}^m$$

Case 1: A invertible Eq. (13.4)

$$\mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b}$$

Unique solution; objective $= 0$.

Case 2: A not invertible — use SVD:

$$\mathbf{A} = \mathbf{U} \begin{bmatrix} \mathbf{T} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{V}^\top \quad \text{Eq. (13.5)}$$

where $\mathbf{U} \in \mathbb{R}^{m \times m}$, $\mathbf{V} \in \mathbb{R}^{n \times n}$ orthogonal, $\mathbf{T} \in \mathbb{R}^{k \times k}$ diagonal (positive).

Minimum-norm solution via pseudoinverse:

$$\mathbf{x}^* = \mathbf{A}^+\mathbf{b} \tag{13.18}$$

$$\mathbf{A}^+ = \mathbf{V} \begin{bmatrix} \mathbf{T}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{U}^\top \tag{13.19}$$

Special forms:

$$\mathbf{A}^+ = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top \quad \text{cols indep.} \tag{13.20}$$

$$\mathbf{A}^+ = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1} \quad \text{rows indep.} \tag{13.21}$$

$$\mathbf{A}^+ = \mathbf{A}^{-1} \quad \mathbf{A} \text{ invertible} \tag{13.22}$$

SVD Derivation of the Pseudoinverse

Substituting SVD into $\|\mathbf{Ax}-\mathbf{b}\|^2$:

$$\|\mathbf{Ax}-\mathbf{b}\|^2 = \left\|\mathbf{U}\begin{bmatrix}\mathbf{T} & \mathbf{0}\\ \mathbf{0} & \mathbf{0}\end{bmatrix}\mathbf{V}^\top\mathbf{x}-\mathbf{b}\right\|^2 \tag{13.6}$$

$$= \left\|\begin{bmatrix}\mathbf{T} & \mathbf{0}\\ \mathbf{0} & \mathbf{0}\end{bmatrix}(\mathbf{V}^\top\mathbf{x})-(\mathbf{U}^\top\mathbf{b})\right\|^2 \tag{13.8}$$

$$= \left\|\mathbf{T}\mathbf{v}_{\text{fixed}}-\mathbf{U}_1^\top\mathbf{b}\right\|^2+\left\|\mathbf{U}_2^\top\mathbf{b}\right\|^2 \tag{13.11}$$

where $\mathbf{U}=[\mathbf{U}_1,\mathbf{U}_2]$, $\mathbf{V}=[\mathbf{V}_1,\mathbf{V}_2]$, $\mathbf{v}_{\text{fixed}}=\mathbf{V}_1^\top\mathbf{x}$.

Optimal Fixed Component — Eq. (13.13)

$$\mathbf{v}_{\text{fixed}}^* = \mathbf{T}^{-1}\mathbf{U}_1^\top\mathbf{b}$$

- $\mathbf{v}_{\text{free}}=\mathbf{V}_2^\top\mathbf{x}$ is unconstrained; setting it to $\mathbf{0}$ gives the **minimum-norm** solution.
- Result: $\mathbf{x}^*=\mathbf{V}\begin{bmatrix}\mathbf{T}^{-1} & \mathbf{0}\\ \mathbf{0} & \mathbf{0}\end{bmatrix}\mathbf{U}^\top\mathbf{b}=\mathbf{A}^+\mathbf{b}$

Python: Pseudoinverse & Least Squares

```python
import numpy as np
from scipy.linalg import lstsq, svd

# Example: A is 3x2 (overdetermined), b is 3-vector
A = np.array([[2., 1.],
              [-4., 3.],
              [1., 1.]])
b = np.array([1., 2., 0.5])

# --- Method 1: pseudoinverse via SVD ---
A_pinv = np.linalg.pinv(A)      # V T^{-1} U^T (eq. 13.19)
x_star = A_pinv @ b
print(f"x* (pinv): {x_star}")   # minimum-norm least-squares solution

# --- Method 2: scipy lstsq (numerically robust, recommended) ---
x_lstsq, residuals, rank, sv = lstsq(A, b)
print(f"x* (lstsq): {x_lstsq}")

# --- Method 3: normal equations (columns linearly independent) ---
x_normal = np.linalg.solve(A.T @ A, A.T @ b)  # eq. (13.20)
print(f"x* (normal eq): {x_normal}")

# Verify residual
print(f"||Ax - b|| = {np.linalg.norm(A @ x_star - b):.6f}")
```

Listing 1: Unconstrained least-squares via pseudoinverse (NumPy/SciPy)

Outline

Least Squares with Linear Inequalities

Problem Statement — Eq. (13.23)

$$\underset{\mathbf{x}}{\text{minimize}} \ \|\mathbf{A}\mathbf{x} - \mathbf{b}\| \qquad \text{subject to} \quad \mathbf{C}\mathbf{x} \geq \mathbf{d}$$

(Less-than inequalities are negated; equality constraints are eliminated.)

Equality constraint elimination via LQ decomposition:

1. Decompose $\mathbf{C}_{\text{EQ}} = \mathbf{L}\mathbf{Q}$ (LQ = QR of transpose)
2. Substitute $\mathbf{x} = \mathbf{Q}^\top \begin{bmatrix} \mathbf{y}_1^* \\ \mathbf{y}_2 \end{bmatrix}$ where $\mathbf{y}_1^* = \mathbf{L}_{11}^{-1}\mathbf{d}_{\text{EQ}}$ Eq. (13.24)
3. Optimization reduces to variable $\mathbf{y}_2$:

Reduced Problem — Eq. (13.26)

$$\underset{\mathbf{y}_2}{\text{minimize}} \ \left\|(\mathbf{A}\mathbf{Q}^\top)_{(\cdot,c+1:n)}\mathbf{y}_2 - \left(\mathbf{b} - (\mathbf{A}\mathbf{Q}^\top)_{(\cdot,1:c)}\mathbf{y}_1^*\right)\right\|$$

$$\text{s.t.} \quad (\mathbf{C}\mathbf{Q}^\top)_{(\cdot,c+1:n)}\mathbf{y}_2 \geq \mathbf{d} - (\mathbf{C}\mathbf{Q}^\top)_{(\cdot,1:c)}\mathbf{y}_1^*$$

Algorithm 13.1 — Solve Quadratic Program (General Form)

Method

Convert QP $\rightarrow$ LeastSquaresWithInequalities, solve, back-substitute. Assumes $\mathbf{C}_{\text{EQ}} \in \mathbb{R}^{c \times n}$ has rank $c < n$.

Steps:

1. Extract $\mathbf{A}, \mathbf{b}, \mathbf{C}_{\text{LE}}, \mathbf{d}_{\text{LE}}, \mathbf{C}_{\text{EQ}}, \mathbf{d}_{\text{EQ}}$
2. Compute LQ decomposition: $\mathbf{C}_{\text{EQ}} = \mathbf{LQ}$
3. Compute $\mathbf{y}_1^* = \mathbf{L}_{11}^{-1}\mathbf{d}_{\text{EQ}}$
4. Form reduced matrices $\mathbf{A}_2 = (\mathbf{AQ}^\top)_{(\cdot, c+1:n)}$, etc.
5. Solve LeastSquaresWithInequalities for $\mathbf{y}_2$
6. Return $\mathbf{x}^* = \mathbf{Q}^\top[\mathbf{y}_1^*; \mathbf{y}_2]$

Complexity

- LQ decomposition: $O(cn^2)$
- Reduced problem size: $(n - c)$ variables
- Overall cost dominated by inner NNLS solver

Key assumption: $\mathbf{C}_{\text{EQ}}$ is full row rank (no redundant/contradictory equality constraints).

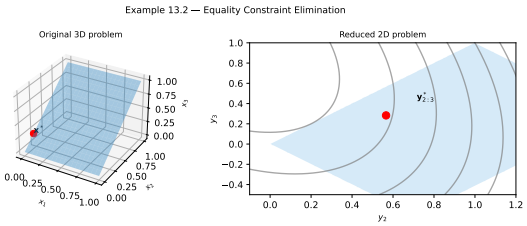
Example 13.2 — 3D QP with Equality Constraint

Original 3D Problem

$$\underset{\mathbf{x}}{\text{minimize}} \left\| \begin{bmatrix} 1 & 2 & 3 \\ -2 & 0 & 0 \end{bmatrix} \mathbf{x} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right\|$$

$$\mathbf{x} \leq \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \ \mathbf{x} \geq \mathbf{0}, \ [-1 \ \ 1 \ \ 1]\mathbf{x} = 0$$

Feasible region: $0 \leq x_1 \leq 1$, $0 \leq x_3 \leq 1$, $x_2 = x_3$



Solution: $\mathbf{x}^* = [0,\ 0.2,\ 0.2]^\top$
After equality elimination: $\mathbf{y}_1 = [0]$, reduced to 2D problem.

Python: Least Squares with Inequalities

```python
import numpy as np
from scipy.optimize import minimize

def solve_qp_lstsq(A, b, C_LE, d_LE, C_EQ=None, d_EQ=None):
    """Solve minimize||Ax - b|| s.t. C_LE x <= d_LE, C_EQ x = d_EQ"""
    n = A.shape[1]
    # Objective: minimize (1/2)||Ax - b||^2
    Q = A.T @ A           # = U^T U  (eq 13.1 with q = -A^T b)
    q = -A.T @ b

    constraints = []
    if C_LE is not None:
        # C_LE x <= d_LE  =>  -(C_LE x - d_LE) >= 0
        constraints.append({'type': 'ineq',
                            'fun': lambda x: d_LE - C_LE @ x})
    if C_EQ is not None:
        constraints.append({'type': 'eq',
                            'fun': lambda x: C_EQ @ x - d_EQ})

    def obj(x):
        return 0.5 * x @ Q @ x + q @ x

    def grad(x):
        return Q @ x + q

    res = minimize(obj, np.zeros(n), jac=grad,
                   method='SLSQP', constraints=constraints)
```

Outline

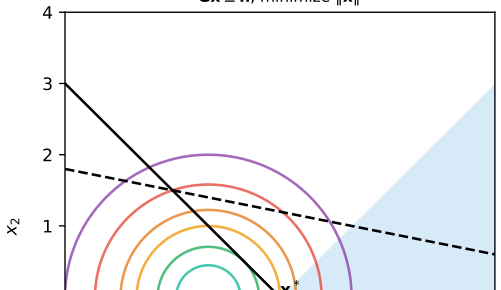
Least Distance Programming

Least Distance Program (LDP) — Eq. (13.27)

$$\underset{\mathbf{x}}{\text{minimize}} \ \|\mathbf{x}\| \qquad \text{subject to} \quad \mathbf{G}\mathbf{x} \geq \mathbf{h}$$

where $\mathbf{G} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{h} \in \mathbb{R}^m$.
Find the point in the feasible set **closest to the origin**.

Figure 13.3 — Least Distance Program
$\mathbf{G}\mathbf{x} \geq \mathbf{h}$, minimize $\|\mathbf{x}\|$



Reduction from Least Squares with Inequalities:

1. Apply SVD: $\mathbf{A} = \mathbf{U} \begin{bmatrix} \mathbf{T} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{V}^\top$
2. Objective becomes $\left\| \mathbf{T}\mathbf{v}_{\text{fixed}} - \mathbf{U}_1^\top \mathbf{b} \right\|^2$
3. Change of variables: $\mathbf{z} = \mathbf{T}\mathbf{v}_{\text{fixed}} - \mathbf{U}_1^\top \mathbf{b}$
4. Minimizing $\|\mathbf{z}\|$ subject to linear inequalities Eq. (13.29–13.31)

Properties:

LDP → NNLS Transformation

The LDP can be solved via a **nonnegative least-squares** (NNLS) problem:

NNLS Form — Eq. (13.32–13.33)

$$\underset{\mathbf{x}}{\text{minimize}} \ \|\mathbf{E}\mathbf{x} - \mathbf{f}\| \qquad \text{subject to} \quad \mathbf{x} \geq \mathbf{0}$$

Form the augmented matrix from LDP data $(\mathbf{G}, \mathbf{h})$:

$$\mathbf{E} = \begin{bmatrix} \mathbf{G}^\top \\ \mathbf{h}^\top \end{bmatrix} \in \mathbb{R}^{(n+1)\times m}, \qquad \mathbf{f} = \begin{bmatrix} \mathbf{0}_n \\ 1 \end{bmatrix} \in \mathbb{R}^{n+1}$$

Variables $\mathbf{y}$ of NNLS are proportional to dual variables of LDP.

Recovery of primal solution:

$$\mathbf{r} = \mathbf{E}\mathbf{y}^* - \mathbf{f} \tag{13.34}$$

$$x_j^* = -\frac{r_j}{r_{n+1}}, \quad j = 1, \ldots, n, \quad r_{n+1} < 0 \tag{13.35}$$

Algorithm 13.3 (in outline):

1. Form $\mathbf{E} = [\mathbf{G}^\top; \mathbf{h}^\top]$, $\mathbf{f} = [\mathbf{0}; 1]$

Python: Least Distance Program via NNLS

```python
import numpy as np
from scipy.optimize import nnls

def solve_least_distance(G, h, eps=1e-12):
    """
    Solve: minimize ||x|| subject to G x >= h
    Returns (x_star, solved).
    """
    m, n = G.shape
    # Build NNLS problem (eq. 13.33): E y = f, y >= 0
    E = np.vstack([G.T, h.T])        # shape (n+1, m)
    f = np.zeros(n + 1)
    f[n] = 1.0                       # last component is 1

    y_star, _ = nnls(E, f)

    r = E @ y_star - f               # residual  (eq. 13.34)
    if r[n] >= -eps:                 # r_{n+1} must be negative
        return np.zeros(n), False    # malformed / infeasible

    x_star = -r[:n] / r[n]           # eq. 13.35
    return x_star, True

# -- Example 13.3 ------------------------------------------------
G = np.array([[2.0, -2.0],
              [-1.0,  5.0]])
h = np.array([2.0, -7.0])
```

Example 13.3 — Solving an LDP

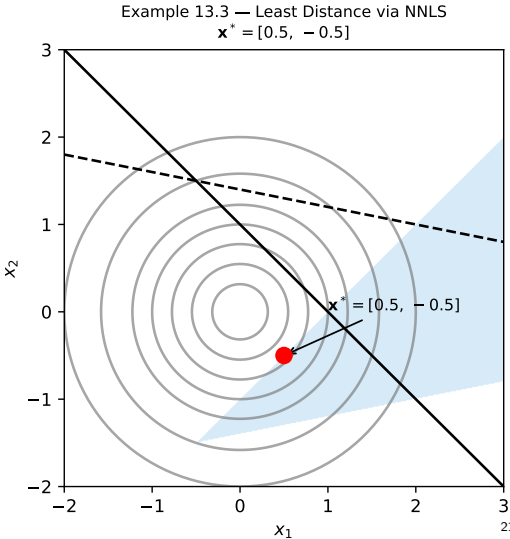
Problem Data

$$\mathbf{G} = \begin{bmatrix} 2.0 & -2.0 \\ -1.0 & 5.0 \end{bmatrix}, \quad \mathbf{h} = \begin{bmatrix} 2.0 \\ -7.0 \end{bmatrix}$$

NNLS matrices:

$$\mathbf{E} = \begin{bmatrix} 2.0 & -1.0 \\ -2.0 & 5.0 \\ 2.0 & -7.0 \end{bmatrix}, \quad \mathbf{f} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

NNLS solution: $\mathbf{y}^* = [0.167,\ 0.000]^\top$
Residual: $\mathbf{r} = [0.333,\ -0.333,\ -0.667]^\top$
$r_{n+1} = r_3 = -0.667 < 0$ ✓
Primal solution: $\mathbf{x}^* = -\frac{\mathbf{r}_{1:2}}{r_3} = [0.500,\ -0.500]^\top$



Outline

NNLS: Problem Setup and Partition

Nonnegative Least-Squares (NNLS) — Eq. (13.32)

$$\underset{\mathbf{x}}{\text{minimize}} \ \|\mathbf{E}\mathbf{x} - \mathbf{f}\| \qquad \text{subject to} \quad \mathbf{x} \geq \mathbf{0}$$

where $\mathbf{E} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{f} \in \mathbb{R}^m$.

Simplex-inspired partition: Assign each index i to $\mathcal{B}$ or $\mathcal{V}$:

$$i \in \mathcal{B} \implies x_i \geq 0 \quad \text{(inactive constraint)} \tag{13.54}$$
$$i \in \mathcal{V} \implies x_i = 0 \quad \text{(active constraint)} \tag{13.55}$$

At a solution $\mathbf{x}^*$:

- $\mathbf{x}_{\mathcal{V}} = \mathbf{0}$
- $\mathbf{x}^*$ minimizes $\|\mathbf{E}_{\mathcal{B}}\mathbf{x} - \mathbf{f}\|$ over $\mathcal{B}$
- Dual variables: $\mu_i \geq 0$ for $i \in \mathcal{V}$, $\mu_i = 0$ for $i \in \mathcal{B}$ with $x_i > 0$

KKT conditions at $\mathbf{x}^*$:

$$\boldsymbol{\mu} = \mathbf{E}^\top(\mathbf{E}\mathbf{x}^* - \mathbf{f})$$

- If $\boldsymbol{\mu} \geq \mathbf{0}$: optimality confirmed
- Else: relax the constraint with most negative μ_i

Algorithm 13.4 — NNLS Solver

Lawson–Hanson Active-Set Algorithm

Initialize: $\mathbf{x} = \mathbf{0}$, $\mathcal{B} = \emptyset$, $\mathcal{V} = \{1, \ldots, n\}$, max $3n$ iterations.

1. Compute $\boldsymbol{\mu} = \mathbf{E}^\top(\mathbf{E}\mathbf{x} - \mathbf{f})$
2. **If** all $\mu_i \geq 0$: **return** $(\mathbf{x}, \texttt{true})$ (KKT satisfied)
3. Select **entering index** $q = \arg\min_{i \in \mathcal{V}} \mu_i$ (most negative)
4. Move q from $\mathcal{V}$ to $\mathcal{B}$; set $\mathbf{E}_{\mathcal{B}}[:, q] = \mathbf{E}[:, q]$
5. **Inner loop** (while $\mathcal{B} \neq \emptyset$ and $\alpha \neq 1$):
   1. $\mathbf{z} = \mathbf{E}_{\mathcal{B}}^{+}\mathbf{f}$ (pseudoinverse solve)
   2. If $z_q \leq 0$: remove q back to $\mathcal{V}$; break
   3. $\alpha = \min\left(1,\ \min_{i : z_i < 0, i \in \mathcal{B}} \frac{x_i}{x_i - z_i}\right)$
   4. $\mathbf{x}_{\mathcal{B}} \leftarrow \mathbf{x}_{\mathcal{B}} + \alpha(\mathbf{z}_{\mathcal{B}} - \mathbf{x}_{\mathcal{B}})$
   5. Zero out any $x_i \leq 0$; move those indices back to $\mathcal{V}$
6. Update $\boldsymbol{\mu} \leftarrow \mathbf{E}^\top(\mathbf{E}\mathbf{x} - \mathbf{f})$; repeat

Guarantee

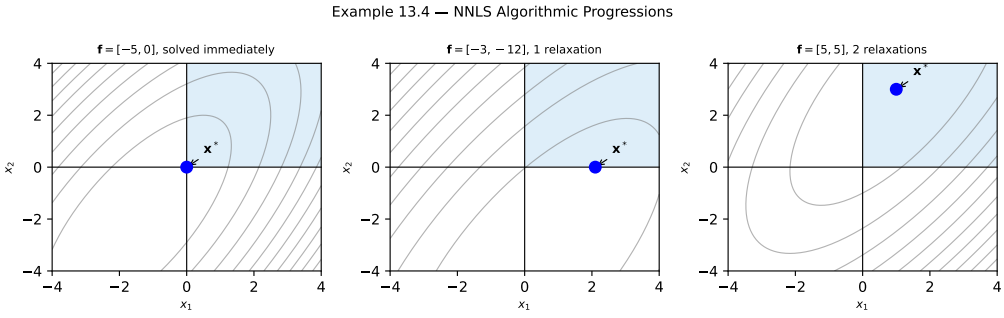
Every NNLS problem has a solution; this algorithm always terminates. Max $3n$ outer iterations.

Python: NNLS Algorithm Implementation

```python
import numpy as np
from scipy.optimize import nnls as scipy_nnls

def solve_nnls(E, f, max_iter=None):
    """
    NNLS: minimize ||Ex - f|| s.t. x >= 0
    Implements Lawson-Hanson active-set (Algorithm 13.4).
    Returns (x, solved).
    """
    m, n = E.shape
    if max_iter is None:
        max_iter = 3 * n
    x = np.zeros(n)
    mu = -E.T @ f           # initial dual estimate
    P = np.zeros(n, dtype=bool)   # True = active in B
    Ep = np.zeros((m, n))
    q_prev = -1

    for _ in range(max_iter):
        if np.all(P) or np.all(mu >= 0):
            return x, True   # success
        # Choose entering index (most negative mu not in B and != q_prev)
        candidates = [(mu[i], i) for i in range(n) if not P[i] and i != q_prev]
        if not candidates:
            break
        q = min(candidates)[1]
        q_prev = q
```

NNLS Algorithmic Progression — Example 13.4

$$\mathbf{E} = \begin{bmatrix} 2.0 & 1.0 \\ -4.0 & 3.0 \end{bmatrix}$$



- $\mathbf{f} = [-5, 0]$: unconstrained minimizer already in $\mathbf{x} \geq \mathbf{0} \rightarrow$ 0 relaxations
- $\mathbf{f} = [-3, -12]$: 1 constraint relaxation needed
- $\mathbf{f} = [5, 5]$: 2 constraint relaxations needed

KKT Proof of NNLS Optimality

Goal: Show that $\mathbf{x}^*$ from Algorithm 13.4 satisfies KKT for the LDP.

Step 1: Feasibility. From KKT, gradient of NNLS objective is balanced by constraints:

$$\mathbf{0} \leq \mathbf{E}^\top \mathbf{r} = [\mathbf{G}\ \mathbf{h}] \begin{bmatrix} \mathbf{x}^* \\ -1 \end{bmatrix} (-r_{n+1}) = (\mathbf{G}\mathbf{x}^* - \mathbf{h}) \|\mathbf{r}\|^2 \tag{13.45}$$

This shows $\mathbf{G}\mathbf{x}^* \geq \mathbf{h}$. ✓

Step 2: Stationarity.

$$\mathbf{x}^* - \mathbf{G}^\top \boldsymbol{\mu} = \mathbf{0} \tag{13.48}$$

$$\mathbf{x}^* = \mathbf{G}^\top \boldsymbol{\mu} \tag{13.49}$$

with $\boldsymbol{\mu} = \mathbf{y}^* \|\mathbf{r}\|^{-2} \geq \mathbf{0}$.

Step 3: Complementary slackness. y_i^* is zero for inactive constraints.

Conclusion

We have a primal-dual pair $(\mathbf{x}^*, \boldsymbol{\mu}^*)$ satisfying all KKT conditions for a convex problem $\Rightarrow \mathbf{x}^*$ is **globally optimal**.

Outline

Dual Problem for QP

Consider the general QP (13.1) with positive-definite $\mathbf{Q}$:

$$\underset{\mathbf{x}}{\text{minimize}} \ \tfrac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{q}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{C}_{\text{LE}}\mathbf{x} \le \mathbf{d}_{\text{LE}},\ \mathbf{C}_{\text{EQ}}\mathbf{x} = \mathbf{d}_{\text{EQ}}$$

Lagrangian — Eq. (13.58)–(13.60)

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = \tfrac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{e}^\top \mathbf{x} + f$$

where $\mathbf{e} = \mathbf{q} + \mathbf{C}_{\text{LE}}^\top \boldsymbol{\mu} + \mathbf{C}_{\text{EQ}}^\top \boldsymbol{\lambda}$ and $f = -\boldsymbol{\mu}^\top \mathbf{d}_{\text{LE}} - \boldsymbol{\lambda}^\top \mathbf{d}_{\text{EQ}}$.

Dual Function — Eq. (13.61)–(13.63)

Minimize $\mathcal{L}$ over $\mathbf{x}$: gradient zero $\Rightarrow \mathbf{x}^* = -\mathbf{Q}^{-1}\mathbf{e}$

$$\mathcal{D}(\boldsymbol{\mu} \ge \mathbf{0}, \boldsymbol{\lambda}) = -\tfrac{1}{2}\mathbf{e}^\top \mathbf{Q}^{-1}\mathbf{e} + f$$

Dual Problem — Eq. (13.64)

$$\underset{\boldsymbol{\mu}, \boldsymbol{\lambda}}{\text{maximize}} \ -\tfrac{1}{2}\mathbf{e}^\top \mathbf{Q}^{-1}\mathbf{e} + f \qquad \text{subject to} \quad \boldsymbol{\mu} \ge \mathbf{0}$$

Dual Certificates — Verifying Optimality

KKT Conditions as Dual Certificate (Sec. 13.6)

Condition	Requirement	Interpretation
Primal Feasibility	$\mathbf{C}_{LE}\mathbf{x} \leq \mathbf{d}_{LE},\ \mathbf{C}_{EQ}\mathbf{x} = \mathbf{d}_{EQ}$	x is in the feasible set
Dual Feasibility	$\boldsymbol{\mu} \geq \mathbf{0}$	Dual variables are non-neg.
Complementary Slackness	$\boldsymbol{\mu}^\top(\mathbf{C}_{LE}\mathbf{x} - \mathbf{d}_{LE}) = 0$	Active iff constraint is tight
Zero Duality Gap	$f(\mathbf{x}^*) = \mathcal{D}(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$	Confirms global optimality

Python: Dual Certificate Verification

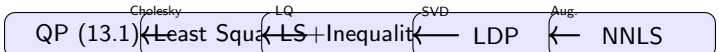
```python
import numpy as np

def verify_kkt(Q, q, C_LE, d_LE, x_star, mu_star, tol=1e-6):
    """
    Verify KKT conditions for QP:
      min (1/2)x^T Q x + q^T x  s.t.  C_LE x <= d_LE
    Parameters: primal x_star, dual mu_star (>= 0)
    """
    # 1. Primal feasibility
    primal_feas = np.all(C_LE @ x_star <= d_LE + tol)

    # 2. Dual feasibility
    dual_feas = np.all(mu_star >= -tol)

    # 3. Stationarity:  Q x + q + C_LE^T mu = 0
    stationarity = np.linalg.norm(Q @ x_star + q + C_LE.T @ mu_star) < tol

    # 4. Complementary slackness:  mu_i * (C_LE x - d_LE)_i = 0
    slack = C_LE @ x_star - d_LE          # should be <= 0
    comp_slack = np.abs(mu_star @ slack) < tol

    return {'primal_feasible': primal_feas,
            'dual_feasible':   dual_feas,
            'stationary':      stationarity,
            'comp_slack':      comp_slack}

# Dual objective value D(mu, lambda)
```

Outline

Chapter 13 Summary

Key Concepts

1. **Quadratic programs** have quadratic objectives and linear constraints.
2. A **convex** QP (positive-definite $\mathbf{Q}$) is equivalent to a least-squares problem.
3. **Unconstrained least squares** is solved exactly via the *pseudoinverse* $\mathbf{A}^{+}\mathbf{b}$.
4. A **general least-squares** problem converts to a LS with linear inequalities via LQ decomposition.
5. LS with inequalities converts to a **least distance program** (LDP).
6. An LDP converts to a **nonneg. least-squares** (NNLS) problem.
7. NNLS is solved by the **Lawson–Hanson active-set** method (Alg. 13.4).
8. **Dual certificates** (KKT conditions + zero duality gap) verify global optimality.



Selected Exercises

Exercise 13.1

Are NNLS problems always feasible?
Yes: $\mathbf{x} \geq \mathbf{0}$ is always satisfiable (e.g., $\mathbf{x} = \mathbf{0}$).

Exercise 13.2

Are LDPs always feasible?
No: $\mathbf{G}\mathbf{x} \geq \mathbf{h}$ may be empty. Example: $\mathbf{1}^\top \mathbf{x} \geq 1$ and $-\mathbf{1}^\top \mathbf{x} \geq 1$ simultaneously.

Exercise 13.3

Do LDPs have unique solutions?
Yes: Suppose two optimal solutions $\mathbf{a}, \mathbf{b}$ with $\|\mathbf{a}\| = \|\mathbf{b}\|$. Then $\mathbf{c} = \frac{1}{2}(\mathbf{a} + \mathbf{b})$ is feasible and $\|\mathbf{c}\| < \|\mathbf{a}\|$ (strict convexity) — contradiction.

Exercise 13.5

WLOG, $\mathbf{Q}$ can be assumed symmetric: $\mathbf{x}^\top \mathbf{Q}\mathbf{x} = \mathbf{x}^\top \mathbf{Q}_{\text{sym}}\mathbf{x}$ where $\mathbf{Q}_{\text{sym}} = \frac{1}{2}(\mathbf{Q} + \mathbf{Q}^\top)$, since the antisymmetric part $\mathbf{Q}_{\text{anti}} = -\mathbf{Q}_{\text{anti}}^\top$ satisfies $\mathbf{x}^\top \mathbf{Q}_{\text{anti}}\mathbf{x} = 0$ for all $\mathbf{x}$.

Exercise 13.6 — QP $\equiv$ Least Squares

Show: $\text{minimize}_{\mathbf{x}} \left\|\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top}\mathbf{q}\right\| \equiv \text{minimize}_{\mathbf{x}} \frac{1}{2}\mathbf{x}^\top\mathbf{Q}\mathbf{x} + \mathbf{q}^\top\mathbf{x}$

$$
\begin{aligned}
\arg\min_{\mathbf{x}} \left\|\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top}\mathbf{q}\right\| &= \arg\min_{\mathbf{x}} \left\|\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top}\mathbf{q}\right\|^2 \\
&= \arg\min_{\mathbf{x}} (\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top}\mathbf{q})^\top(\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top}\mathbf{q}) \\
&= \arg\min_{\mathbf{x}} \mathbf{x}^\top\mathbf{U}^\top\mathbf{U}\mathbf{x} + \mathbf{q}^\top\mathbf{U}^{-1}\mathbf{U}\mathbf{x} + \mathbf{x}^\top\mathbf{U}^\top\mathbf{U}^{-\top}\mathbf{q} + \mathbf{q}^\top\mathbf{U}^{-1}\mathbf{U}^{-\top}\mathbf{q} \\
&= \arg\min_{\mathbf{x}} \mathbf{x}^\top\mathbf{Q}\mathbf{x} + 2\mathbf{q}^\top\mathbf{x} + \mathbf{q}^\top\mathbf{Q}^{-1}\mathbf{q} \\
&= \arg\min_{\mathbf{x}} \tfrac{1}{2}\mathbf{x}^\top\mathbf{Q}\mathbf{x} + \mathbf{q}^\top\mathbf{x}
\end{aligned}
$$

(constant $\mathbf{q}^\top\mathbf{Q}^{-1}\mathbf{q}$ dropped; factor of 2 absorbed into $\mathbf{q}^\top\mathbf{x}$.)

Complete Python Pipeline — Example 13.1

```python
import numpy as np
from scipy.optimize import minimize, LinearConstraint

# Example 13.1: minimize ||Ax - b|| s.t. 2x1 - x2 >= 2, x1 - 3x2 <= 4
A = np.array([[2., 1.], [-4., 3.]])
b_vec = np.array([1., 2.])

# Quadratic form: Q = A^T A,  q = -A^T b
Q = A.T @ A
q = -A.T @ b_vec

def objective(x):
    return 0.5 * x @ Q @ x + q @ x

def gradient(x):
    return Q @ x + q

# Constraints: 2x1 - x2 >= 2  =>  [2, -1] x >= 2
#              x1 - 3x2 <= 4  =>  -x1 + 3x2 >= -4
C = np.array([[ 2., -1.],   # 2x1 - x2 >= 2
              [-1.,  3.]])   # x1 - 3x2 <= 4 (negated)
d = np.array([2., -4.])

constraint = LinearConstraint(C, lb=d, ub=np.inf)

result = minimize(objective, np.zeros(2), jac=gradient,
                  method='SLSQP', constraints={'type': 'ineq',
```

Chapter 13: Quadratic Programming

Algorithms for Optimization, 2nd ed.

Kochenderfer & Wheeler, MIT Press, 2026

What we covered:

- QP formulation and geometry
- Unconstrained LS via pseudoinverse
- Equality/inequality elimination
- Least Distance Programs
- NNLS: Lawson–Hanson algorithm
- Dual certificates & KKT

Python tools used:

- `numpy.linalg.pinv` — pseudoinverse
- `scipy.linalg.lstsq` — least squares
- `scipy.optimize.nnls` — NNLS
- `scipy.optimize.minimize` (SLSQP) — general QP
- `scipy.linalg.svd` — SVD decomposition